

Compression and Optimal Codes

Prefix codes, code length, and Huffman's algorithm

Prof. Nipun Batra

Mathematical Foundations for AI · IIT Gandhinagar

**Give short names to common
outcomes**

A five-symbol source

Suppose a source emits

symbol	probability
A	0.40
B	0.20
C	0.15
D	0.15
E	0.10

A fixed-length binary code needs three bits per symbol because $2^2 < 5 \leq 2^3$.

Fixed-length code

One valid assignment is

$A \rightarrow 000$, $B \rightarrow 001$, $C \rightarrow 010$,

$D \rightarrow 011$, $E \rightarrow 100$.

Every symbol costs three bits, so the average length is

$L = 3$ bits/symbol.

The code ignores the fact that A is four times as common as E.

Variable-length code

Try

$A \rightarrow 0, \quad B \rightarrow 111, \quad C \rightarrow 101,$

$D \rightarrow 110, \quad E \rightarrow 100.$

The expected length is

$$L = 0.40(1) + 0.20(3) + 0.15(3) + 0.15(3) + 0.10(3).$$

$$L = 2.20 \text{ bits/symbol.}$$

Encode one message

Message: A B A E

Fixed-length code:

000 001 000 100 — 12 bits.

Variable-length code:

0 111 0 100 — 8 bits.

The decoder must be able to determine where one codeword ends and the next begins. The code structure must make those boundaries recoverable from the bit stream.

An ambiguous code

Suppose

$A \rightarrow 0$, $B \rightarrow 01$, $C \rightarrow 10$.

The bit string 010 can be decoded as

1. $A \ C = 0 \mid 10$, or
2. $B \ A = 01 \mid 0$.

The problem is that the codeword for A is a prefix of the codeword for B.

Prefix-free codes

A code is **prefix-free** if no codeword is the prefix of another codeword.

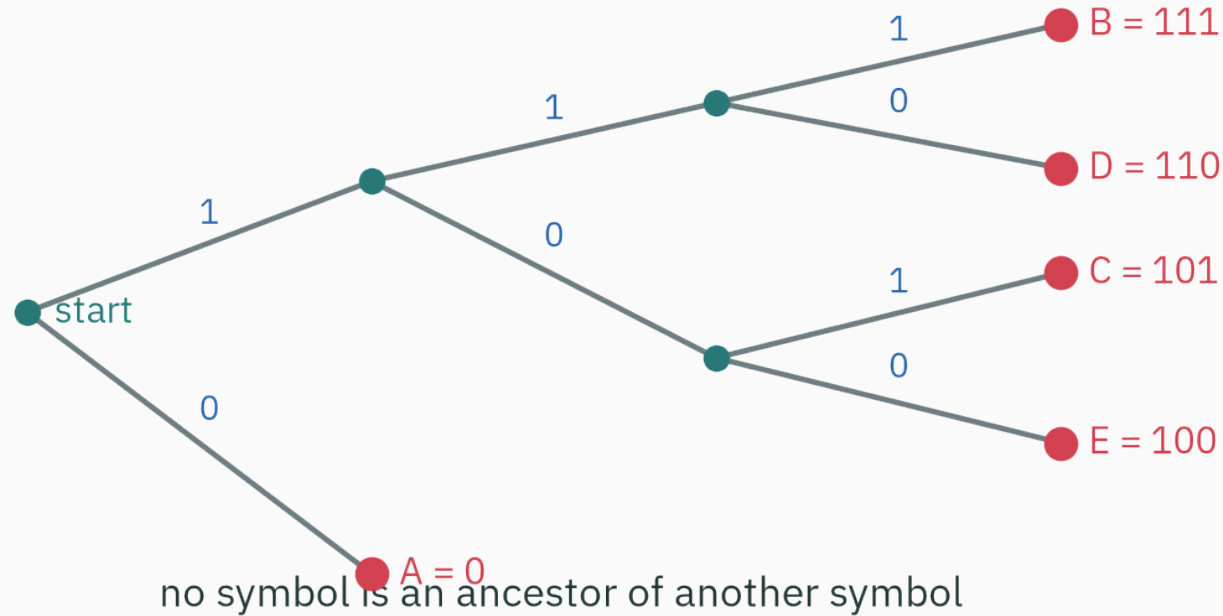
Then decoding can proceed from left to right without separators.

Prefix-free is sufficient, but not necessary, for unique decoding. For example, $A \rightarrow 0, B \rightarrow 01$ is uniquely decodable but not prefix-free; prefix-free codes are preferred because they decode immediately from left to right.

In a binary tree:

1. each left edge writes 0,
2. each right edge writes 1,
3. symbols occupy leaves only.

A prefix tree



Reaching a leaf completes one symbol; decoding restarts at the root.

Learning outcomes

By the end of this lecture you will be able to:

1. distinguish fixed-length, uniquely decodable, and prefix-free codes,
2. relate probability to ideal code length $-\log_2 p$,
3. use the Kraft inequality as a feasibility check,
4. construct ★ a Huffman tree by repeated merging,
5. calculate expected code length and compare it with entropy,
6. explain why Huffman coding is optimal among binary prefix codes,
7. and connect probabilistic prediction with compression.

Code length follows surprise

Ideal length

L22 assigned surprise

$$I(x) = -\log_2 p(x).$$

This suggests an ideal code length

$$\ell(x) = -\log_2 p(x).$$

Examples:

$$p = \frac{1}{2} \rightarrow \ell = 1, \quad p = \frac{1}{4} \rightarrow \ell = 2, \quad p = \frac{1}{8} \rightarrow \ell = 3.$$

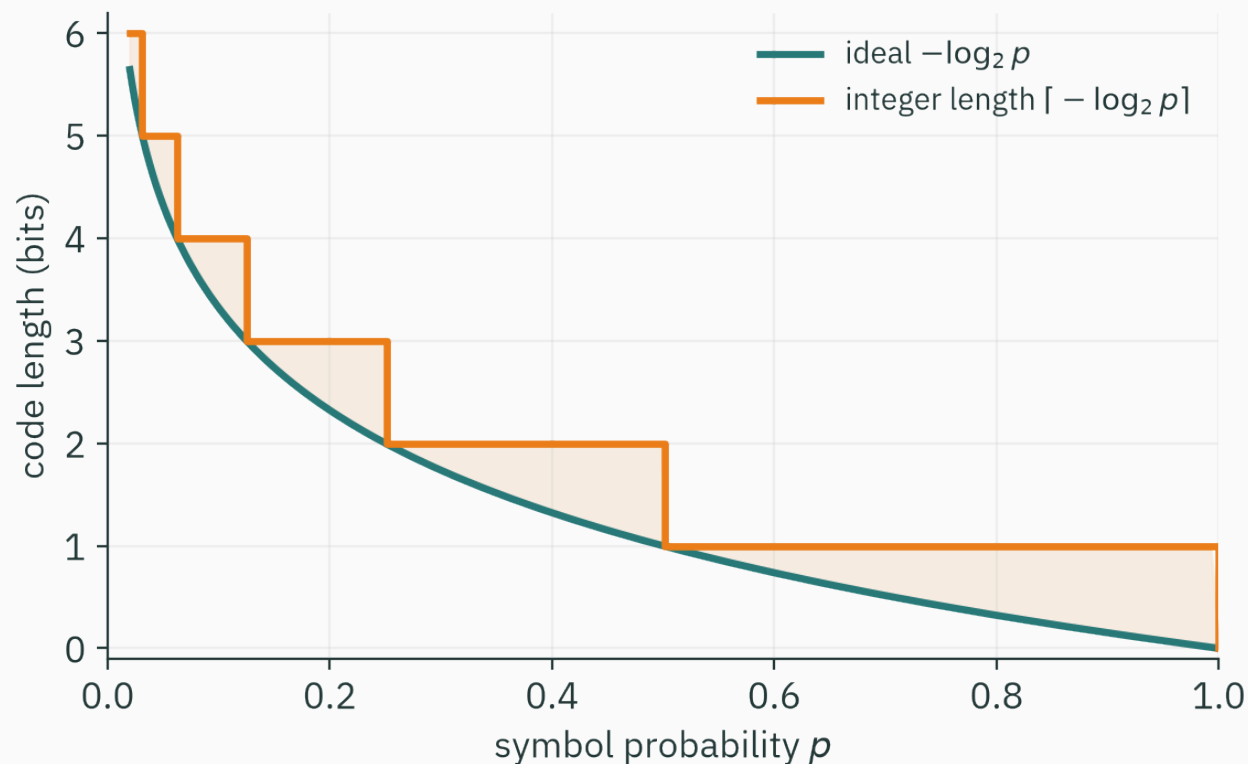
Ideal lengths need not be integers

For $p = 0.40$,

$-\log_2 0.40 \approx 1.322$ bits.

A single binary codeword cannot contain 1.322 bits.

Integer code lengths introduce a rounding cost. Coding long blocks or using arithmetic coding can approach fractional average lengths.



Choosing $\ell_i = \lceil -\log_2 p_i \rceil$ adds less than one bit to each ideal length.

The Kraft inequality

Integer lengths $\ell_1, \dots, \ell_K$ can form a binary prefix code if and only if

$$\sum_{i=1}^K 2^{-\ell_i} \leq 1.$$

Each codeword at depth ℓ_i occupies a fraction $2^{-\ell_i}$ of the leaves at a common deeper level.

Kraft check: a valid set of lengths

For lengths $(1, 3, 3, 3, 3)$,

$$2^{-1} + 4 \times 2^{-3} = \frac{1}{2} + \frac{4}{8} = 1.$$

The tree is exactly filled. This is the length pattern used for the five-symbol variable code.

Kraft check: an impossible set

For lengths $(1, 2, 2, 2)$,

$$2^{-1} + 3 \times 2^{-2} = \frac{1}{2} + \frac{3}{4} = 1.25 > 1.$$

There is not enough room in a binary tree for one one-bit leaf and three two-bit leaves.

Expected code length

For probabilities p_i and lengths ℓ_i ,

$$L = \sum_i p_i \ell_i.$$

Entropy is the corresponding average ideal length:

$$H(X) = \sum_i p_i (-\log_2 p_i).$$

For any binary prefix code,

$$L \geq H(X).$$

Shannon lengths give an upper bound

Choose

$$\ell_i = \lceil -\log_2 p_i \rceil.$$

Since $\lceil a \rceil < a + 1$,

$$\ell_i \leftarrow \log_2 p_i + 1.$$

Average both sides:

$$H(X) \leq L < H(X) + 1.$$

Which set of binary code lengths can satisfy the Kraft inequality?

A. (1, 1, 2)

B. (1, 2, 3, 3)

C. (1, 2, 2, 2)

D. (1, 1, 1)

Correct: B — (1, 2, 3, 3)

Why: $\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{8} = 1$. Every other option has Kraft sum greater than one.

**Merge the two least frequent
symbols**

The greedy rule

Huffman's algorithm operates on symbol weights:

1. take the two smallest weights,
2. merge them into one node whose weight is their sum,
3. return the merged node to the collection,
4. repeat until one root remains,
5. label left and right edges 0 and 1.

Less frequent symbols are pushed deeper in the tree.

Why merge the two smallest?

In an optimal full binary tree, two deepest leaves can be made siblings.

Placing the two least probable symbols at those deepest positions cannot increase expected length.

Merge those siblings into a single pseudo-symbol. The remaining problem has one fewer symbol and the same form.

This exchange argument plus induction proves optimality.

Counts in “MATHEMATICS”

M A T H E M A T I C S contains 11 letters.

A	2	M	2
T	2	C	1
E	1	H	1
I	1	S	1

There are eight distinct symbols, so a fixed code needs three bits per letter.

The merge sequence

Ties can be resolved in several ways; all optimal trees have the same expected length.

One deterministic sequence of weights is

$$1 + 1 \rightarrow 2,$$

$$1 + 1 \rightarrow 2,$$

$$1 + 2 \rightarrow 3,$$

$$2 + 2 \rightarrow 4,$$

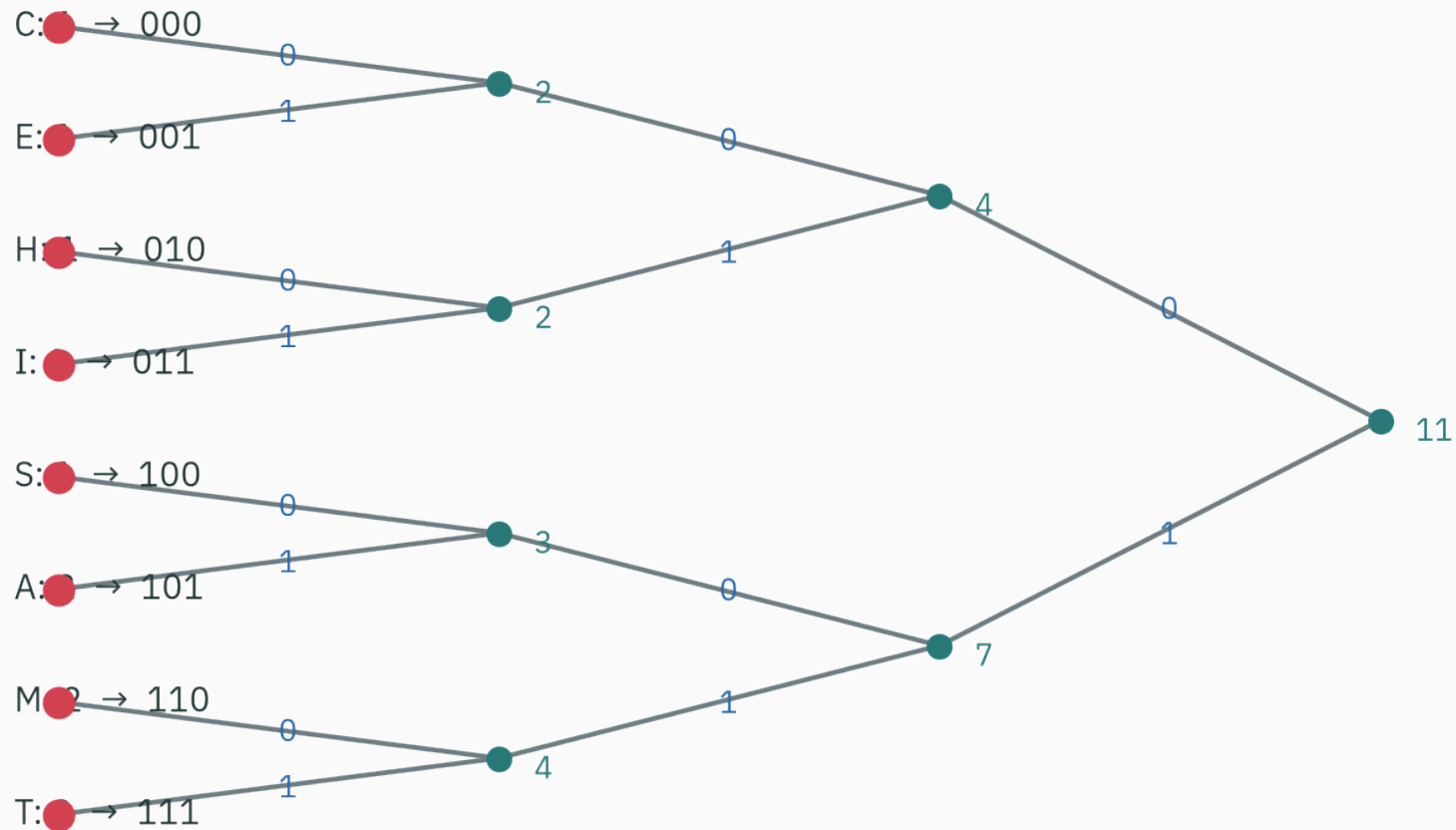
$$2 + 2 \rightarrow 4,$$

$$3 + 4 \rightarrow 7,$$

$$4 + 7 \rightarrow 11.$$

The resulting tree

Huffman tree for "MATHEMATICS" (11 letters)



Here every letter happens to receive length three. Huffman cannot improve on the fixed-length code for this short frequency table.

Verify the encoded word

One generated code is

$A \rightarrow 101, \quad C \rightarrow 000, \quad E \rightarrow 001, \quad H \rightarrow 010,$

$I \rightarrow 011, \quad M \rightarrow 110, \quad S \rightarrow 100, \quad T \rightarrow 111.$

Every codeword has length three, so “MATHEMATICS” uses

$11 \times 3 = 33$ bits.

The example is still useful: an optimal code need not beat fixed length on every finite sample.

A source with a useful imbalance

Return to probabilities

$(0.40, 0.20, 0.15, 0.15, 0.10)$.

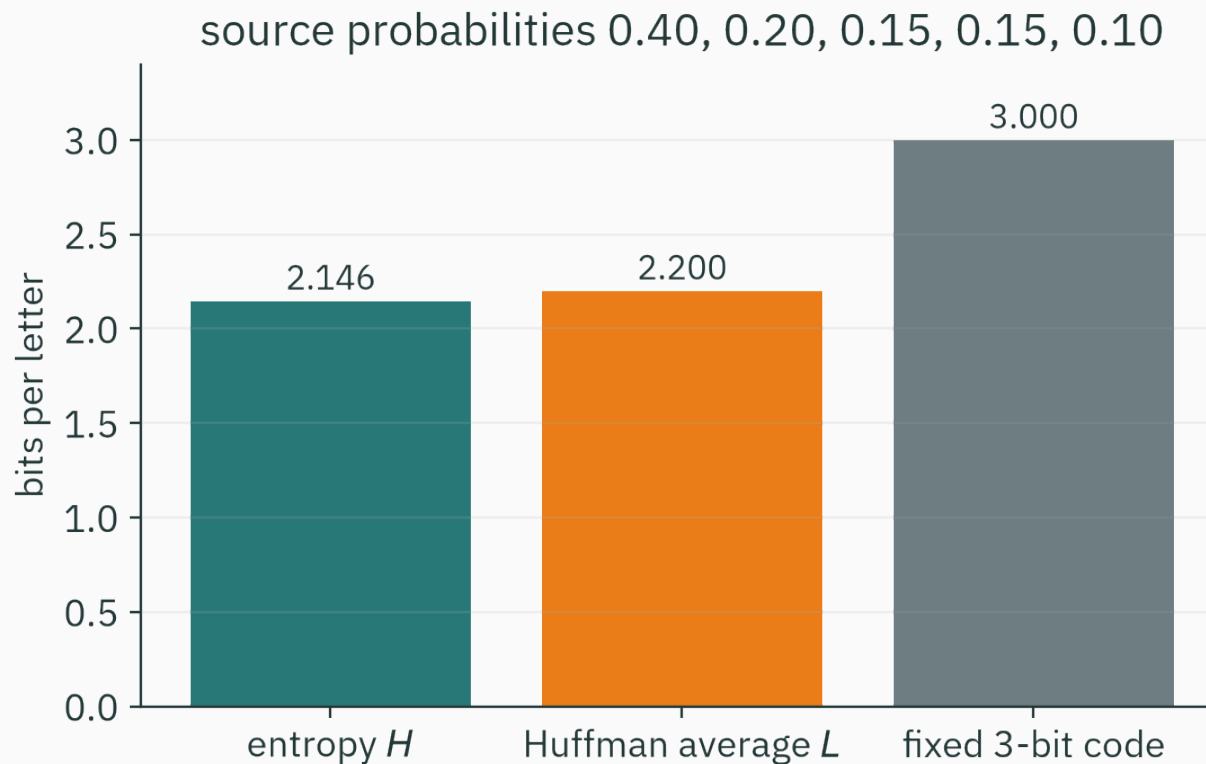
Huffman produces lengths

$(1, 3, 3, 3, 3)$

and one code assignment

$A \rightarrow 0, B \rightarrow 111, C \rightarrow 101, D \rightarrow 110, E \rightarrow 100$.

Compare entropy and lengths



$$H = 2.146 \leq L_{\text{Huffman}} = 2.200 < 3 = L_{\text{fixed}}.$$

Huffman is optimal within a class

Huffman minimizes expected length among binary prefix codes for a known symbol distribution.

It does not mean:

1. the encoded file is always shorter after storing the tree,
2. one symbol at a time reaches entropy exactly,
3. the assumed probabilities match future data,
4. context has been used.

Those are separate modeling and engineering questions.

A small compressor

A priority queue implements the merges

```
import heapq, itertools

serial = itertools.count()
heap = [(count, next(serial), symbol)
        for symbol, count in counts.items()]
heapq.heapify(heap)

while len(heap) > 1:
    wa, _, a = heapq.heappop(heap)
    wb, _, b = heapq.heappop(heap)
    heapq.heappush(heap, (wa+wb, next(serial), (a, b)))

tree = heap[0][2]
```

Traverse the tree to assign codes

```
def make_codes(node, prefix="", out=None):
    out = {} if out is None else out
    if isinstance(node, str):
        out[node] = prefix or "0"
    else:
        left, right = node
        make_codes(left, prefix + "0", out)
        make_codes(right, prefix + "1", out)
    return out
```

Encode by table lookup

```
def encode(text, code):  
    return "".join(code[ch] for ch in text)  
  
bits = encode("MATHEMATICS", code)  
print(bits)  
print(len(bits))
```

Real implementations pack bits into bytes; a Python string of '0' and '1' is only a teaching representation.

Decode by walking the tree

```
def decode(bits, tree):
    output, node = [], tree
    for bit in bits:
        node = node[0] if bit == "0" else node[1]
        if isinstance(node, str):
            output.append(node)
            node = tree
    assert node is tree      # no unfinished codeword
    return "".join(output)
```

What must be stored?

A standalone compressed file needs enough metadata to reconstruct the code:

1. the tree or canonical code lengths,
2. the number of valid data bits in the final byte,
3. the original alphabet or symbol ordering,
4. and often checksums or format information.

For short messages, this overhead can exceed the saved bits.

The decoder does not need the exact tree shape if it receives the code length of each symbol.

Canonical Huffman coding assigns codewords deterministically from sorted lengths.

Benefits:

1. compact header,
2. fast table-based decoding,
3. reproducible code assignment.

Many file formats use this representation.

**Better probabilities give
shorter descriptions**

Code with the wrong frequency model

If a compressor expects distribution q but data come from p , it chooses lengths near

$$-\log_2 q(x).$$

Its average cost under the true source is

$$\mathbb{E}_{[X \sim p]}[-\log_2 q(X)].$$

This is cross-entropy. L24 will show exactly how much larger it is than $H(p)$.

Independent-letter model

L22 estimated letter frequencies without context.

Such a model assigns one probability to each letter regardless of its neighbors:

$$q(x_1, \dots, x_n) = \prod_{t=1}^n q(x_t).$$

It cannot exploit that u is likely after q, or that common words recur.

Conditional prediction

A context model uses

$$q(x_1, \dots, x_n) = \prod_{t=1}^n q(x_t \mid x_{<t}).$$

If the context makes the next symbol predictable, $q(x_t \mid x_{<t})$ is larger and

$$-\log_2 q(x_t \mid x_{<t})$$

is shorter.

Compression improves when prediction improves.

Huffman assigns an integer number of bits to each symbol. Arithmetic coding represents an entire sequence as an interval.

It can approach the fractional sequence length

$$-\log_2 q(x_1, \dots, x_n)$$

much more closely, especially when some probabilities are near one.

Modern learned compressors combine a probability model with an arithmetic or range coder.

Language models report coding cost

A language model outputs a probability distribution for the next token.

For observed tokens $x_1, \dots, x_n$, its total ideal code length is

$$-\sum_{t=1}^n \log_2 q(x_t \mid x_{<t}).$$

Divide by n to obtain bits per token. Exponentiate to obtain perplexity in L24.

**Compression and error
correction solve different
problems**

Compression removes predictable bits

Source coding uses probability to reduce average description length.

Channel coding adds structured redundancy so corrupted bits can be detected or corrected.

The two goals point in opposite directions:

1. compress before transmission,
2. then add carefully designed redundancy for the channel.

Repetition code

Encode one bit as three:

$0 \rightarrow 000, \quad 1 \rightarrow 111.$

If at most one bit flips, majority vote recovers the original:

$111 \rightarrow 101 \rightarrow \text{decode as } 1.$

The price is rate $\frac{1}{3}$: three transmitted bits per information bit.

A limit also exists for noisy channels

★ OPTIONAL

Shannon's channel coding theorem states that reliable communication is possible below a channel's capacity and impossible above it, under precise assumptions.

Entropy governs source description; mutual information and capacity govern transmission.

We will use mutual information only briefly in L24.

From entropy to an actual code

Frequent symbols receive short codewords; Huffman's greedy merges produce the optimal binary prefix tree.

1. Prefix-free codes decode without separators.
2. Kraft's inequality characterizes feasible prefix-code lengths.
3. Ideal length is $-\log_2 p$; entropy is the average ideal length.
4. Huffman coding minimizes expected length among binary prefix codes.
5. For a source distribution, $H \leq L_{\text{Huffman}} < H + 1$.
6. A better conditional predictor gives a shorter description.

Practice

1. Check Kraft's inequality for lengths $(2, 2, 2, 2)$ and $(1, 3, 3, 3)$.
2. Build a Huffman code for weights $(8, 4, 2, 1, 1)$.
3. Compute the expected length of codes $A : 0, B : 10, C : 110, D : 111$ under probabilities $(\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8})$.
4. Explain why storing a Huffman tree can make a tiny file larger.

1. The Kraft sums are 1 and $\frac{1}{2} + \frac{3}{8} = \frac{7}{8}$; both are feasible.
2. One optimal code has lengths (1, 2, 3, 4, 4); the two weight-1 symbols are deepest siblings.
3. $L = \frac{1}{2}(1) + \frac{1}{4}(2) + \frac{1}{8}(3) + \frac{1}{8}(3) = 1.75$ bits/symbol, equal to the entropy.
4. The tree or length table, padding, alphabet, and format metadata create overhead independent of message length.

Next: the cost of a wrong model

Entropy is the average cost under the true distribution. A model usually supplies a different distribution.

Next lecture derives

$$H(p, q) = H(p) + D_{\text{KL}}(p \parallel q).$$

The excess code length becomes KL divergence, and minimizing cross-entropy becomes maximum likelihood.

Probabilities determine ideal lengths; a code turns those lengths into bits.